

TLS Pie / Kizim Robotics - End-to-End Bench Test Checklist

Use this checklist to verify the full chain: Velodyne lidar to .pcap capture, and MicroView to Raspberry Pi signaling. Work through the stages in order. Each stage only adds one new variable, so a failure tells you exactly where the problem is. Package version: SETUP_PACKAGE_18.07.26.

Stage 0: Static code validation (already completed on 2026-07-18)

- Python scripts compiled cleanly: VLPbuttons.py, VLPwaitbutton.py, VLPstatussignal.py (both Pi and MicroView package copies)
- Shell scripts passed bash -n syntax check: VLPrecord.sh, VLPselfcheck.sh, setup_tls_pie_pi.sh
- LidarHDMicroviewV1.0.ino brace/paren/bracket balance verified
- Result: PASS. No syntax errors found. This does not confirm hardware timing or GPIO wiring.

Stage 1: Pi environment self-check (no MicroView or lidar needed)

Run on the Pi:

```
sudo Pi/TLS-Pie/Raspberry\ Pie4/TLS-Pie/VLPselfcheck.sh
```

Confirm:

- python3, tcpdump, ip, bash found
- Python can import RPi.GPIO
- VLPrecord.sh, VLPbuttons.py, VLPwaitbutton.py, VLPstatussignal.py are executable
- Network interface exists and has an IPv4 address

Result: PASS / FAIL. Notes:

Stage 2: Dry-run install

Run on the Pi:

```
sudo TLSPIE_DRY_RUN=1 bash Pi/setup_tls_pie_pi.sh
```

Confirm:

- The printed actions match expectations (packages, autostart entry, permissions, target directory)
- No real changes are made in dry-run mode

Result: PASS / FAIL. Notes:

Stage 3: MicroView to Pi signaling only (no motor movement, no lidar connected)

Flash LidarHDMicroviewV1.0.ino to the MicroView. Wire only the trigger and status lines (Arduino D7 to Pi GPIO17, Arduino D8 to Pi GPIO27, Pi GPIO22 through level shifter to MicroView D4, common ground). Run on the Pi:

```
sudo Pi/TLS-Pie/Raspberry\ Pie4/TLS-Pie/VLPrecord.sh
```

Press a scan button on the MicroView and confirm:

- OLED shows READY, then WAIT PI, then REC

- /tmp/tlspie/VLPrecord.status shows RECORDING
- Pressing kill/stop shows STOPPED or DONE on the OLED
- /tmp/tlspie/VLPrecord.status shows COMPLETED

Result: PASS / FAIL. Notes:

Stage 4: Add the lidar, capture-only test

Connect the Velodyne lidar to the Pi. Trigger a short capture from the MicroView. Confirm:

- A new .pcap file appears in ~/velodyne
- File size is greater than zero
- `tcpdump -r ~/velodyne/TLS\_\*.pcap -c 20` shows UDP packets from the lidar IP only, not ARP, DHCP, or mDNS noise

Result: PASS / FAIL. Notes:

Stage 5: Full bench scan with motor engaged

Follow the safe bench-test order: power and ground, Arduino boot and display, button behavior, D7/D8 trigger outputs, Pi GPIO17/GPIO27 input, stepper driver control pins D3/D5/D6, tcpdump capture output. Start with the shortest cycle (180 degrees at 1 deg/sec). Confirm:

- Motor moves only after the MicroView shows REC (that is, only after the Pi has confirmed tcpdump is running)
- Motor completes the expected rotation and returns to start
- A complete, non-empty .pcap file is produced at the end

Result: PASS / FAIL. Notes:

Stage 6: MATLAB conversion validation

Copy the resulting .pcap to the PC. Run it through TLS_Velodyne_Converter.mlapp or TLS_Velodyne_script.m. Confirm:

- totalScanFrames and usableFrames printed to the console are sane for the scan duration
- The previewed point cloud is not empty or garbled
- The merged .ply file opens correctly in CloudCompare or another point cloud viewer

Result: PASS / FAIL. Notes:

Sign-off

- Tester name:
- Date:
- Overall result: PASS / FAIL
- Follow-up items: